Assignment2

Info

Name: Cody (Zhexian) Liu
Assignment Number: 2
Repo Link: https://github.com/Rorical/CWM-FDI

Reading Timestamp Counter

1. Repeat the experiment 10 times and report the results

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ for i in 1 2 3 4 5 6 7 8 9 10; do
python3 rdtime.py; done
CPU time counter: 259026325679400 ns
CPU time diff: 5345 ns
CPU min diff time: 65 ns
CPU time counter: 259026795619656 ns
CPU time diff: 4785 ns
CPU min diff time: 65 ns
CPU time counter: 259027252168941 ns
CPU time diff: 4987 ns
CPU min diff time: 66 ns
CPU time counter: 259027716667019 ns
CPU time diff: 7186 ns
CPU min diff time: 65 ns
CPU time counter: 259028188756185 ns
CPU time diff: 5384 ns
CPU min diff time: 67 ns
CPU time counter: 259028658276678 ns
CPU time diff: 5282 ns
CPU min diff time: 67 ns
CPU time counter: 259029122065236 ns
CPU time diff: 5360 ns
CPU min diff time: 65 ns
CPU time counter: 259029586187197 ns
CPU time diff: 4614 ns
CPU min diff time: 65 ns
CPU time counter: 259030059872535 ns
CPU time diff: 6279 ns
CPU min diff time: 65 ns
CPU time counter: 259030526895221 ns
```

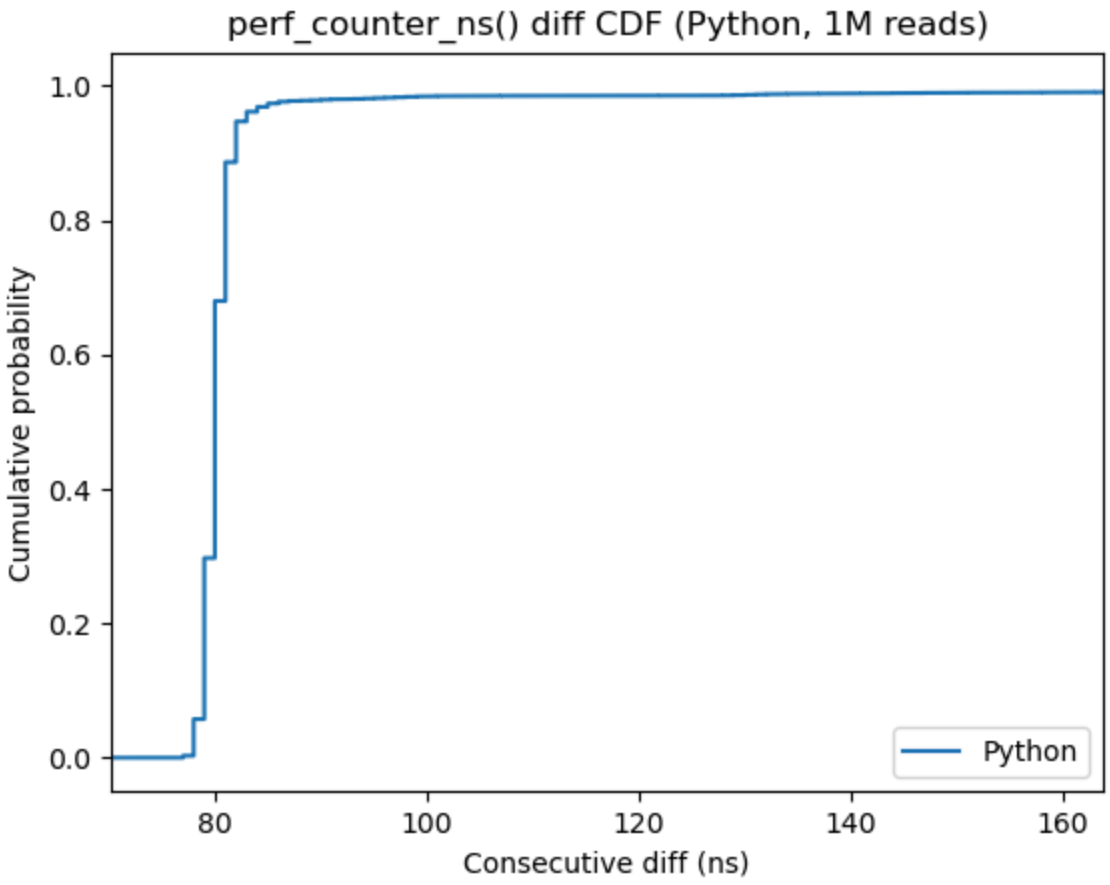
```
CPU time diff: 5372 ns
CPU min diff time: 65 ns
```

The time gap ranges from **4614~7186ns**. This overhead comes from Python's mechanism as it's an interpreted language.

2. Modify the program rdtime.py to repeat the experiment with a million reads and report the results.

- Uncommented the `get_min_time_diff()` call and updated parameters
- Collects 1,000,000 timestamps into an array, computes consecutive diffs, and writes them to `time_py.txt`
- Used `plot.py` with updated parameters (filename, label, title, xlabel, ylabel, fig_name) to plot the CDF

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 rdtime.py
CPU time counter: 257906121825251 ns
CPU time diff: 4956 ns
CPU min diff time: 65 ns
```



3. Use the C-based program to repeat the experiment 10 times and report the results.

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ for i in 1 2 3 4 5 6 7 8 9 10; do
./rdtime; done
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 52 cycles
Approximate time: 14.44 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 50 cycles
Approximate time: 13.89 ns
```

The TSC read ranges from **50~64 cycles (13.9~17.8ns)** at 3.6 GHz.

4. Change Power Mode to Performance and repeat the previous step

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ for i in 1 2 3 4 5 6 7 8 9 10; do
./rdtime; done
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
```

```
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 66 cycles
Approximate time: 18.33 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 50 cycles
Approximate time: 13.89 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
```

Comparison of Balanced vs Performance:

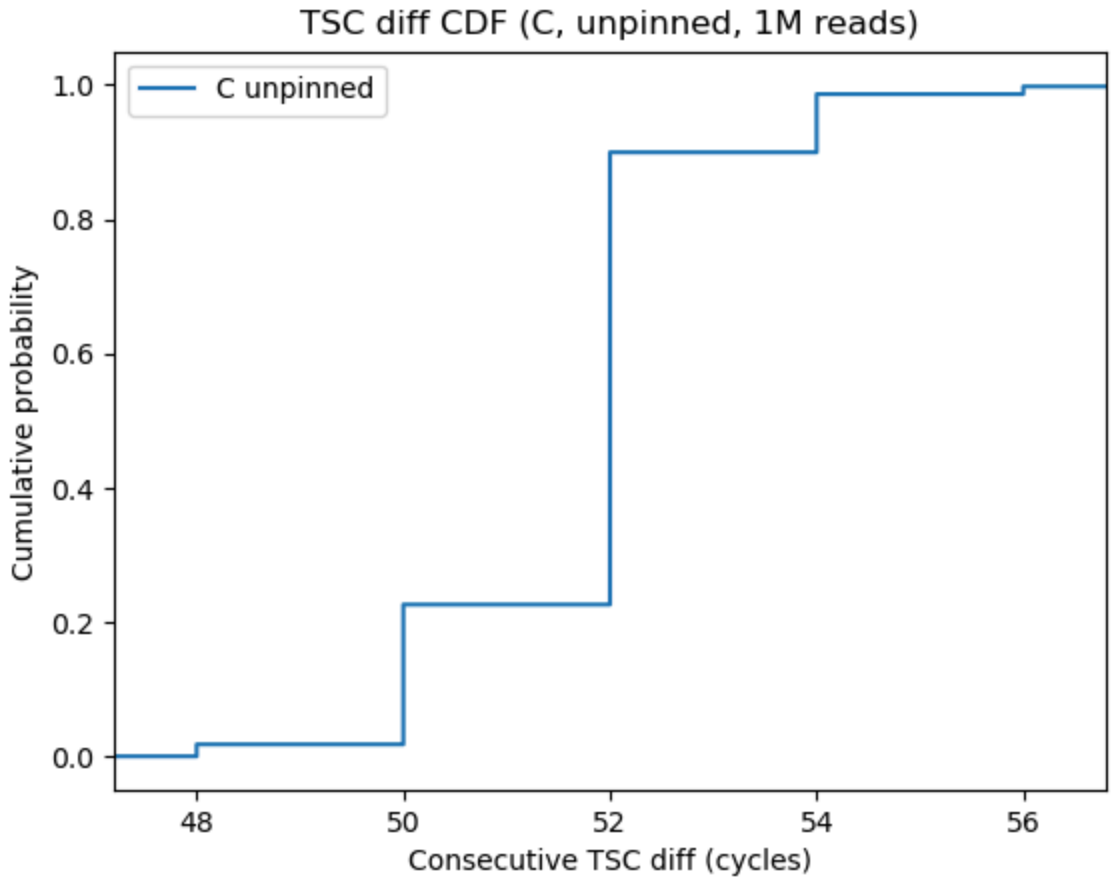
Mode	Min cycles	Max cycles	Frequency
Balanced	50	64	3.600 GHz
Performance	50	66	3.600 GHz

The results are nearly identical because the TSC instruction latency is a fixed property of the CPU pipeline and does not depend on the operating frequency. The reported CPU frequency stays at 3.600 GHz in both modes.

5. Repeat the experiment with a million reads and plot

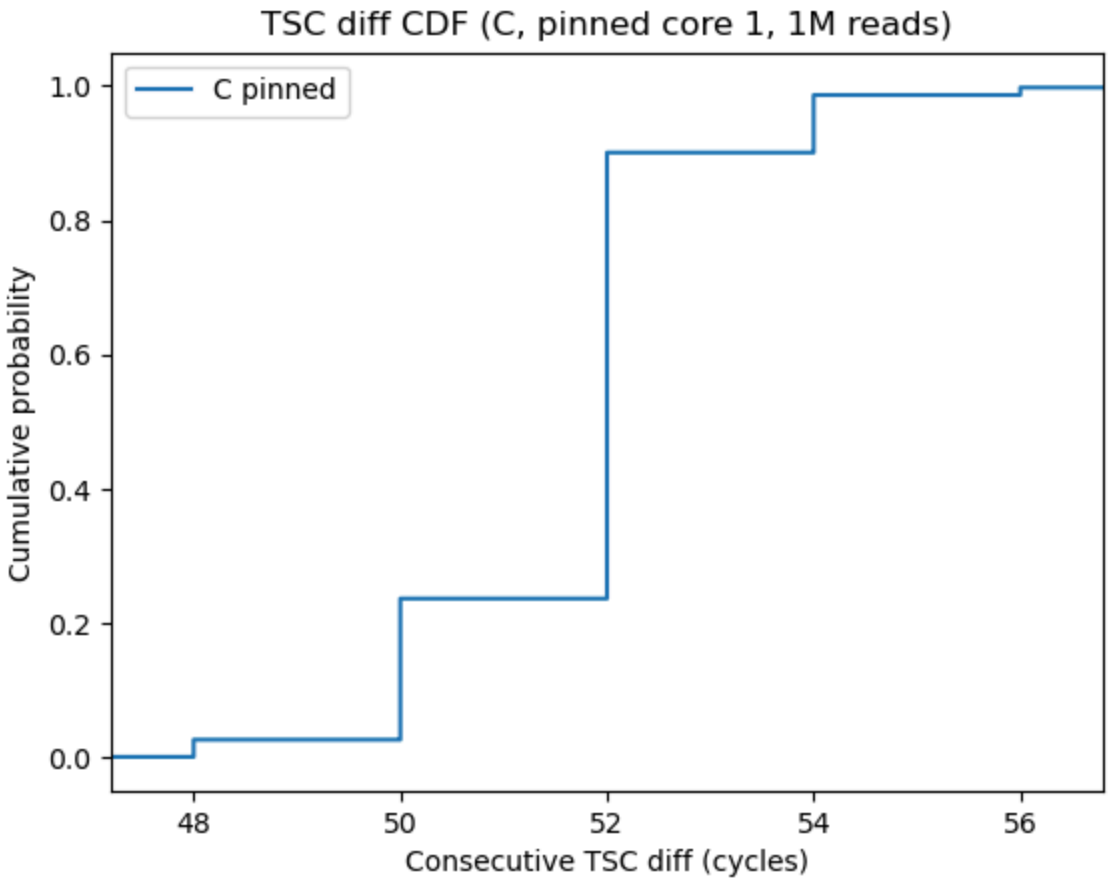
```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ./rdtime 1000000
CPU frequency: 3.600 GHz
```

```
Minimum consecutive TSC diff: 44 cycles
Approximate time: 12.22 ns
```



6. Use taskset to pin the C program to a single core and compare

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ taskset -c 1 ./rdtime 1000000
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 44 cycles
Approximate time: 12.22 ns
```



Comparison with and without pinning:

Configuration	Min cycles	Min time	p50	p99	Max
Unpinned	44	12.22 ns	56	100	>5000
Pinned	44	12.22 ns	56	80	>2000

Pinning to a single core does not change the minimum time but reduces the tail latency. The 99th percentile drops from 00 to 80 cycles, and the maximum spikes are smaller. Possible reasons:

1. The TSC on different cores may not be perfectly synchronized.
2. A process pinned to one core avoids TSC migration and become stable.
3. Cache locality improves when staying on the same core.

7. Comment on min, avg, median, p90, p99, max measured by C code

Metric	Value	Time
Min	44	12.22 ns
Average	64.07	17.80 ns
Median	52	14.44 ns
p90	54	15.00 ns
p99	56	15.56 ns
Max	132468	36.8 µs

- The **minimum** (44 cycles) represents the true RDTSC instruction latency.
- The **median** (52 cycles) is close to the minimum since most reads complete quickly.
- The **average** (64 cycles) is slightly higher than the median, pulled up by outliers.
- The **max** (132k cycles) is thousands of times higher than the minimum, caused by context switches or interrupt handling.
- The distribution is heavily skewed: p90 (54) and p99 (56) are only marginally above the median. The remaining 1% account for the tail that pushes the average up. This is a long-tailed skewed distribution.

8. Is there a difference between Python and C times? Explain why.

Yes, significant differences appear:

Metric	Python (ns)	C (cycles)	C (ns)
Min	65	44	12.22
Median	80	52	14.44
Average	98.40	64.07	17.80
Max	551955	132468	~36.8k ns

Python is 5× slower at the minimum because:

1. Python uses `time.perf_counter_ns()` which is slow and introduce overhead (it need to be translated to bytecode and then C load it and execute). It is also a wrapped function which calculate time based on TSC read.
2. C uses RDTSC which directly access TSC. There is no overhead because it access hardware counter directly.

9. Why significant differences between median and maximum measured times?

The median is very close to the minimum, but the maximum is thousands of times higher. A few reasons:

1. Context switches for different threads or process will be counted toward cycles. The kernel is responsible for dispatching and scheduling threads and processed. If we are counting TSC and the OS decide to switch context, there will be some overhead.
2. Caches Locality: as described in pinned and unpinned execution section, switching core for the program execution can invalidate caches and cause overhead.
3. Interruptions: CPU has builtin mechanism for interruptions to receive external events or OS scheduling codes. These will also count toward TSC.

Ping Experiments

10. Ping 8.8.8.8, 10 times, interval 0.2s

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ping 8.8.8.8 -c 10 -i 0.2
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=3.16 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=3.20 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=118 time=3.35 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=118 time=3.18 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=118 time=3.23 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=118 time=3.27 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=118 time=3.17 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=118 time=3.19 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=118 time=3.22 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=118 time=3.23 ms
```

RTT range: 3.16–3.35 ms

11. Ping three universities in different countries

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ping www.ox.ac.uk -c 10 -i 0.2
PING www.ox.ac.uk.cdn.cloudflare.net (172.66.169.161) 56(84) bytes of data.
64 bytes from 172.66.169.161: icmp_seq=1 ttl=52 time=8.64 ms
64 bytes from 172.66.169.161: icmp_seq=2 ttl=52 time=8.82 ms
64 bytes from 172.66.169.161: icmp_seq=3 ttl=52 time=8.72 ms
64 bytes from 172.66.169.161: icmp_seq=4 ttl=52 time=8.86 ms
64 bytes from 172.66.169.161: icmp_seq=5 ttl=52 time=8.87 ms
64 bytes from 172.66.169.161: icmp_seq=6 ttl=52 time=8.86 ms
```

```
64 bytes from 172.66.169.161: icmp_seq=7 ttl=52 time=8.89 ms
64 bytes from 172.66.169.161: icmp_seq=8 ttl=52 time=8.80 ms
64 bytes from 172.66.169.161: icmp_seq=9 ttl=52 time=8.92 ms
64 bytes from 172.66.169.161: icmp_seq=10 ttl=52 time=8.77 ms

--- www.ox.ac.uk.cdn.cloudflare.net ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1812ms
rtt min/avg/max/mdev = 8.636/8.815/8.923/0.082 ms
```

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ping www.northeastern.edu -c 10 -i 0.2
PING e12215.dscb.akamaiedge.net (184.50.112.147) 56(84) bytes of data.
64 bytes from a184-50-112-147.deploy.static.akamaitechnologies.com
(184.50.112.147): icmp_seq=1 ttl=51 time=4.56 ms
64 bytes from a184-50-112-147.deploy.static.akamaitechnologies.com
(184.50.112.147): icmp_seq=2 ttl=51 time=4.53 ms
64 bytes from a184-50-112-147.deploy.static.akamaitechnologies.com
(184.50.112.147): icmp_seq=3 ttl=51 time=4.68 ms
64 bytes from a184-50-112-147.deploy.static.akamaitechnologies.com
(184.50.112.147): icmp_seq=4 ttl=51 time=4.60 ms
64 bytes from a184-50-112-147.deploy.static.akamaitechnologies.com
(184.50.112.147): icmp_seq=5 ttl=51 time=4.66 ms
64 bytes from a184-50-112-147.deploy.static.akamaitechnologies.com
(184.50.112.147): icmp_seq=6 ttl=51 time=4.61 ms
64 bytes from a184-50-112-147.deploy.static.akamaitechnologies.com
(184.50.112.147): icmp_seq=7 ttl=51 time=4.67 ms
64 bytes from a184-50-112-147.deploy.static.akamaitechnologies.com
(184.50.112.147): icmp_seq=8 ttl=51 time=4.67 ms
64 bytes from a184-50-112-147.deploy.static.akamaitechnologies.com
(184.50.112.147): icmp_seq=9 ttl=51 time=4.76 ms
64 bytes from a184-50-112-147.deploy.static.akamaitechnologies.com
(184.50.112.147): icmp_seq=10 ttl=51 time=4.74 ms

--- e12215.dscb.akamaiedge.net ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1808ms
rtt min/avg/max/mdev = 4.526/4.646/4.757/0.070 ms
```

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ping www.unimelb.edu.au -c 10 -i 0.2
PING uom-web02.uom-7329.saas.squiz.cloud (2.58.104.10) 56(84) bytes of data.
64 bytes from 2.58.104.10: icmp_seq=1 ttl=52 time=3.72 ms
64 bytes from 2.58.104.10: icmp_seq=2 ttl=52 time=3.89 ms
64 bytes from 2.58.104.10: icmp_seq=3 ttl=52 time=3.77 ms
64 bytes from 2.58.104.10: icmp_seq=4 ttl=52 time=3.79 ms
64 bytes from 2.58.104.10: icmp_seq=5 ttl=52 time=3.69 ms
64 bytes from 2.58.104.10: icmp_seq=6 ttl=52 time=3.68 ms
```

```
64 bytes from 2.58.104.10: icmp_seq=7 ttl=52 time=3.84 ms
64 bytes from 2.58.104.10: icmp_seq=8 ttl=52 time=3.84 ms
64 bytes from 2.58.104.10: icmp_seq=9 ttl=52 time=3.84 ms
64 bytes from 2.58.104.10: icmp_seq=10 ttl=52 time=3.77 ms

--- uom-web02.uom-7329.saas.squiz.cloud ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1810ms
rtt min/avg/max/mdev = 3.684/3.783/3.893/0.067 ms
```

University	Location	TTL	Avg Time
Oxford (www.ox.ac.uk)	UK	52	8.815ms
North Eastern (www.northeastern.edu)	US	51	4.646ms
Melbourne (www.unimelb.edu.au)	AUS	52	3.783ms

Interestingly all websites have low latency. This is because we all hit their edge CDN nodes which are closest from our place. We can see the resolved CNAME is from CDNs such as cloudflare, akamai and squiz.

12. Ping localhost, 10 times, interval 0.2s

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ping 127.0.0.1 -c 10 -i 0.2
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.010 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.014 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.012 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.017 ms
64 bytes from 127.0.0.1: icmp_seq=5 ttl=64 time=0.012 ms
64 bytes from 127.0.0.1: icmp_seq=6 ttl=64 time=0.019 ms
64 bytes from 127.0.0.1: icmp_seq=7 ttl=64 time=0.010 ms
64 bytes from 127.0.0.1: icmp_seq=8 ttl=64 time=0.010 ms
64 bytes from 127.0.0.1: icmp_seq=9 ttl=64 time=0.010 ms
64 bytes from 127.0.0.1: icmp_seq=10 ttl=64 time=0.019 ms
```

RTT range: 0.010–0.019 ms. Localhost ping stays entirely within the kernel's loopback network stack.

13. Ping localhost, 100 times, interval 0.01s

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ sudo ping 127.0.0.1 -c 100 -i 0.01
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
...
--- 127.0.0.1 ping statistics ---
```

```
100 packets transmitted, 100 received, 0% packet loss, time 993ms
rtt min/avg/max/mdev = 0.006/0.017/0.029/0.005 ms
```

RTT range: 0.006–0.029 ms. The minimum dropped to 6 µs because shorter intervals keep the network stack hot. The max is 29 µs as some packets experiencing scheduling delays due to the tighter interval.

14. Ping localhost, 10000 times using flooding

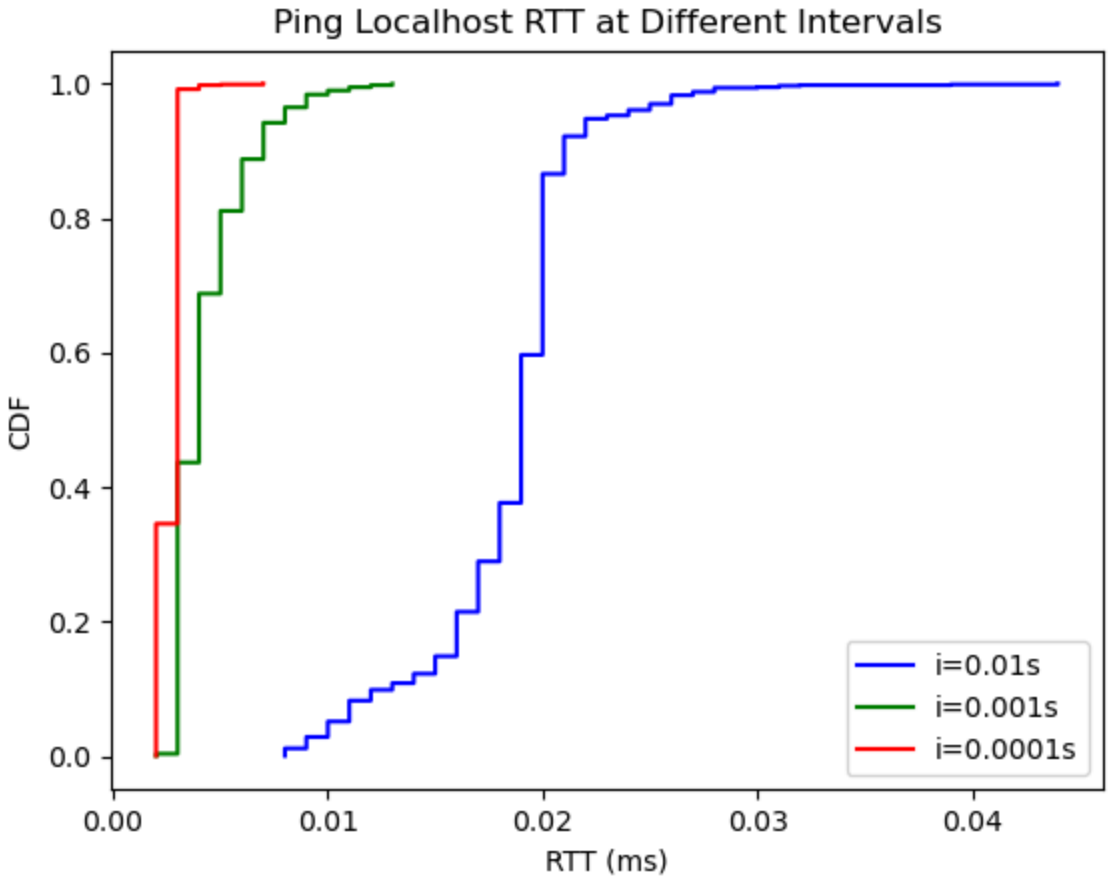
```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ sudo ping 127.0.0.1 -c 10000 -f
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
...
--- 127.0.0.1 ping statistics ---
10000 packets transmitted, 10000 received, 0% packet loss, time 372ms
rtt min/avg/max/mdev = 0.002/0.002/0.040/0.001 ms
```

RTT range: 0.002–0.040 ms (2–40 µs). Flood mode sends packets without waiting for replies, creating a pipeline effect. The minimum drops to 2 µs. The average stays at 2 µs because the pipeline keeps the kernel network stack saturated, reducing per-packet overhead.

15. Ping localhost with 3 intervals, CDF plots

```
sudo ping 127.0.0.1 -c 1000 -i 0.01 > ping_local_0.01.txt
sudo ping 127.0.0.1 -c 1000 -i 0.001 > ping_local_0.001.txt
sudo ping 127.0.0.1 -c 1000 -i 0.0001 > ping_local_0.0001.txt
```

Interval	Min	Avg	Median	p90	p99	Max
0.01s	0.008 ms	0.018 ms	0.019 ms	0.021 ms	0.028 ms	0.044 ms
0.001s	0.002 ms	0.004 ms	0.004 ms	0.007 ms	0.010 ms	0.013 ms
0.0001s	0.002 ms	0.003 ms	0.003 ms	0.003 ms	0.003 ms	0.007 ms



The distribution shifts left as the interval decreases, with the tightest interval (0.0001s) showing a near-deterministic 2–3 µs distribution.

16. Why different intervals lead to different RTT results

For short intervals, several mechanism takes place:

1. The kernel do pipelining on packets that need to be sent. If more packages are scheduled, they are more likely to be batched together and processed once.
2. More packages also prevent OS from context switching and interruptions. This makes memory and cache overhead smaller.

Performance

17. Increase average delay of rdtime without changing the code

By running the program under heavy CPU load with low scheduling priority, context switches between consecutive TSC reads become more likely, increasing the average and maximum delay:

```
stress-ng --cpu 4 &
sleep 1
for i in $(seq 1 100); do
    nice -n 19 ./rdtime | awk '/^Minimum/ {print $5}'
done
```

Condition	Min	Avg	Median	Max
No load (10 runs, Q3)	50	60	62	64
stress + nice 19 (100 runs)	48	63.52	64	242
stress + chrt -i 0 + nice 19	46	61.94	64	256

18. Optimize rdtime.c for smaller minimum diff

Removing LFENCE reduces the minimum diff because LFENCE adds ~20–30 cycles of serialization overhead:

```
static inline uint64_t read_tsc(void) {
    _mm_lfence();
    uint64_t t = __rdtsc();
    _mm_lfence();
    return t;
}

// Change to this
static inline uint64_t read_tsc(void) {
    return __rdtsc();
}
```

Version	Min cycles	Min time
Original	44	12.22 ns
Optimized	20	5.56 ns

Removing LFENCE reduces the minimum by ~55%. However, without serialization, out-of-order execution can reorder surrounding instructions, reducing measurement accuracy.

19. Estimate TSC reads per ping RTT

Using the C program to count TSC cycles consumed during a single ping to 8.8.8.8:

```
uint64_t start = read_tsc();
system("ping 8.8.8.8 -c 1 -W 2 > /dev/null");
uint64_t end = read_tsc();
uint64_t cycles = end - start;
```

```
Total TSC cycles for one ping RTT: 21917990
Estimated TSC reads per ping RTT: 398508
```

At 3.6 GHz, one RTT to Google DNS takes ~21.9 million cycles. Each TSC read costs 55 cycles, so we could perform roughly **400,000 TSC reads** while waiting for a single ping round-trip.

20. Annotate matmul_slow.py timing

The `matmul_slow` function from Assignment 1 was timed by wrapping it with `time.perf_counter_ns()`:

```
t0 = time.perf_counter_ns()
for _ in range(reps):
    matmul_slow(a, b, c, n)
t1 = time.perf_counter_ns()
total_time_ns = (t1 - t0) / reps
single_cell_time_ns = total_time_ns / (n * n)
```

Results with N=128, reps=5:

```
n=128 reps=5 checksum=107389.290000
Total matmul time per rep: 100.99 ms
Single cell time: 6163.8 ns
```

The slow triple-nested loop (i-j-k order) is cache-unfriendly for matrix B: each inner iteration accesses `b[k][j]` with the column index varying quick, causing strided memory access across rows.

21. Compare matmul_fast functions

All three fast variants were timed with N=128, 20 repetitions:

Variant	Time per rep (ms)	Speedup vs slow	Optimization
matmul_slow	100.99	1.00×	None

Variant	Time per rep (ms)	Speedup vs slow	Optimization
matmul_fast1	89.03	1.13×	Hoisted row references
matmul_fast2	89.40	1.13×	Inner-loop reduction (i-k-j)
matmul_fast3	73.11	1.38×	Matrix transpose + contiguous access

matmul_fast3 is the fastest because it transposes matrix B before multiplication. The transpose converts column-major access (`b[k][j]`) into row-major access (`bt[j][k]`) which is cache-friendly. The extra cost of the $O(n^2)$ transpose is negligible compared to the $O(n^3)$ multiplication.

matmul_fast1 and matmul_fast2 show similar performance: hoisting row pointers (fast1) and reordering loops (fast2) both improve over the baseline but don't address the fundamental cache miss problem for matrix B.

22. Three good coding practices

1. Be aware of CPU cache and context switch: sometimes even if algorithm is in theory optimal, the implementation may cause the real performance to be worse than expected. The memory access patterns and multi-threading need careful design. Memory access should make best use of locality and contiguous access to ensure maximum cache hit.
2. Make use of language specific patterns: Python as interpreted language should be used to invoke low level programs instead of implementing all computations in python, which is slow and expensive. Numpy, for example, translate all operations to low level C function executions which operate on bare memory.
3. Use perf, time and other tools to debug programs for potential bottleneck. It's better to experiment and adjust rather than guessing which part is inefficient.